

Aspect-Oriented Design in Java/AspectJ and Ruby

Dean Wampler
Object Mentor, Inc.
dean@objectmentor.com

Abstract

This tutorial teaches professional developers design principles for “production-quality”, aspect-oriented software, written in Java/AspectJ and Ruby.

The discussion starts with a review of the rationale for aspect-oriented software development (AOSD), how it solves some key problems in enterprise applications, followed by a discussion of tool options and techniques for both languages.

After reviewing some of the problems developers encountered during early attempts at aspect-oriented design (AOD), recent strategies for addressing those problems are discussed. The majority of the tutorial then covers extensions to standard object-oriented principles and patterns that support designing and implementing aspect software that is agile, robust, maintainable, testable, and reusable. Guidelines for when to use aspects vs. other techniques are covered. The tutorial concludes with a look forward to ways that aspects could improve architectures and frameworks.

1. Level of the Tutorial

Intermediate to advanced.

2. Prerequisites for Participants

Experience designing and implementing software written in a static language like Java or C# and/or a dynamic language like Ruby or Python. Experience in Object-Oriented Software Development (OOSD) required. A basic understanding of AOSD and AspectJ helpful, but not required.

3. Required Equipment for Participants

Students should bring a laptop (they can be shared) with the following software installed: Java 1.5+, AspectJ 1.5+, an editing environment for Java (Eclipse + AJDT plug-in recommended), Ruby 1.8.4+, plus a

Ruby editing environment (e.g., Eclipse RadRails plug-in or IntelliJ IDEA with ruby plug-in).

4. Description

The tutorial starts with a motivation for AOSD, using a simple example that demonstrates design problems that are not well modularized by objects, because of “cross-cutting concerns” (CCCs), which are multiple “domain” models that intersect in an application (e.g., the primary problem domain and “infrastructure” domains like persistence, security, etc.). The example is presented in both Java and Ruby.

Aspects are defined, followed by a discussion of how they restore system modularity problems raised by CCCs.

With this motivation for AOSD, we explore the nature of Java, a representative static language, and Ruby, a representative dynamic language, as they help or inhibit solving the modularity problems of CCCs using aspects.

We discuss how Java’s metaprogramming facilities are somewhat limited compared with Ruby’s. We examine how this limits Java’s ability to solve CCC problems “natively”, which motivated the emergence of AspectJ, in part. We then explore how AspectJ and pure-Java AOP solutions, like Spring AOP and JBoss AOP implement aspects.

In contrast, Ruby has more powerful metaprogramming support, making it easier to solve at least some CCC problems without AO-specific extensions.

We examine some of these Ruby techniques and see how they are used in *Ruby on Rails* in an “AO-like” way.

This leads to the question of whether or not you really even need “native” AO support if you have good metaprogramming tools. We argue that you do. In particular, we draw an analogy with OOSD; there is an important advantage to having native support for a design approach, even if you can “fake” it with metaprogramming.

We then return to design by discussing early aspect-oriented design (AOD) approaches for aspects, based

on the concept of *obliviousness*, and the problems these approaches created. These problems actually reflect forgotten lessons learned over the years in OOD, such as the importance of minimizing coupling and maximizing the use of abstractions.

Then we review more recent work that addresses these problems, primarily by extending OOD concepts of abstractions with AOD-specific ideas.

With this background, we explore several well-known, object-oriented design principles from an aspect-oriented design perspective.

The design principles include 5 class and interface principles with the acronym SOLID and 6 packaging principles. The SOLID principles are the following:

4.1 Single Responsibility Principle

A class should have only one reason to change.

4.2 Open-Closed Principle

A class should be closed for modification, but open for extension.

4.3 Liskov Substitution Principle

A class “B” is derived from another class “A” when instances of “B” can be substituted for instances of “A” without breaking client expectations.

4.4 Interface Segregation Principle

Interfaces should be as narrow as possible (the SRP for interfaces).

4.5 Dependency Inversion Principle

Abstractions should not depend on details; details should depend on abstractions.

These principles are examined in several ways: (i) how aspects change their interpretations, (ii) how aspects should obey the principles, both in their internal design and when extending behaviors of objects, and (iii) how aspects make it easier to support these principles.

Of the 6 package principles, 3 are relevant for our discussion. They are:

4.6 The Common Closure Principle

The classes in a package should be closed together against the same kinds of changes. A change that affects a closed package affects all the classes in that package and no other packages.

4.7 The Stable Dependencies Principle

Depend in the direction of stability.

4.8 The Stable Abstractions Principle

A package should be as abstract as it is stable.

When considering aspects, several extensions to these principles arise.

4.9 The Updated Open-Closed Principle

A class or aspect should be closed for source and *contract* modification, but open for extension.

4.10 The Updated Liskov Substitution Principle

Derived types or aspects + base types must be substitutable for base types.

4.11 Advice Substitution Principle

Aspect *advice* must obey the contract of the *join points* it advises.

4.12 Introduction Substitution Principle

An introduction must conform to the class contract.

4.13 Pointcut Inversion Principle

Pointcuts should not depend on details; details should depend on abstractions.

4.14 Pointcut Scope Principle

The larger a pointcut’s scope, the more abstract it should be.

(The AOD terms *advice*, *join point*, and *pointcut* will be defined in the tutorial.)

Along the way, we also consider several design patterns, both well-known OO patterns and new AO-specific patterns.

To conclude the tutorial, we explore ways that aspects are likely to be more influential over time in large-scale software design.

We discuss how aspects overcome some of the limitations of OO-based frameworks, specifically the difficulty of adopting most frameworks incrementally or adopting pieces à la carte. This is due to the dependencies between components in traditional OO frameworks, which aspects can help reduce.

The tutorial ends with a discussion of *domain-specific languages* (DSLs), which are especially popular in Ruby. We discuss how DSLs can be used to encapsulate CCC intuitively and how aspects can be used internally to implement the required “weaving” of concerns. We consider Ruby on Rail’s *ActiveRecord* library as an example.

Throughout the tutorial, examples and exercises are used to reinforce the principles discussed.